

SecureLLMBench: A Traceable Framework for Safety and Security Evaluation of Local Large Language Models

Anonymous IEEE-Style Research Draft

Abstract

SecureLLMBench is a research-oriented software framework for conducting traceable evaluations of local large language models (LLMs) under benign, isolated-attack, sequential-attack, and recovery conditions. The framework persists versioned inputs, generation settings, target-model metadata, detector evidence, structured judge measurements, metric artifacts, and explicit failure states. This paper documents the implemented architecture, a local engineering trace, and one real external calibration artifact. The JailbreakBench judge-comparison calibration contains 300 source records and 295 usable records; the fixed Gemma-based mapping achieved 53.9% raw agreement and Cohen's kappa 0.211. This negative result shows that the configured judge cannot replace independent human outcome labels. BSDA, RC, and SAEA have no Phase 1 empirical records and are reported as pending rather than plotted. The contribution is therefore a reproducible measurement and governance substrate, not a claim that a model is secure or that any proposed metric is validated.

Index Terms—LLM security evaluation, reproducibility, local inference, benchmark provenance, behavioral measurement, scientific safeguards.

I. INTRODUCTION

Security evaluation for LLMs requires more than a prompt and a score. A defensible result must identify the benchmark population, model version, generation settings, prompt sequence, measurements, evaluator behavior, and all unavailable prerequisites. SecureLLMBench addresses this engineering requirement by treating an evaluation as an auditable trace rather than as an opaque number. It connects dataset ingestion, local inference, Layer 1 detector evidence, Layer 2 structured judging, behavioral metrics, persistent records, a FastAPI backend, and a research dashboard.

The central design choice is semantic restraint. Layer 1 outputs are detector-native evidence rather than attack-success truth. Layer 2 outputs are structured measurements rather than calibrated probabilities. Missing, failed, uncalibrated, undefined, and not-applicable values remain distinct values in storage and in the dashboard. This prevents a user interface or downstream report from implying that a missing measurement is evidence of zero risk.

A. Contributions

First, the framework implements an end-to-end local evaluation trace with database persistence, read-only API endpoints, and dashboard exploration. Second, it implements four-component BSDA computation, Recovery Capability (RC), and Sequential Attack Evaluation Algorithm (SAEA) runtimes with explicit applicability contracts. Third, it preserves DRAA and PRI as uncalibrated evidence profiles, deliberately withholding scalar risk and model-ranking values. Fourth, it provides reproducibility artifacts, safety tests, and explicit research blockers.

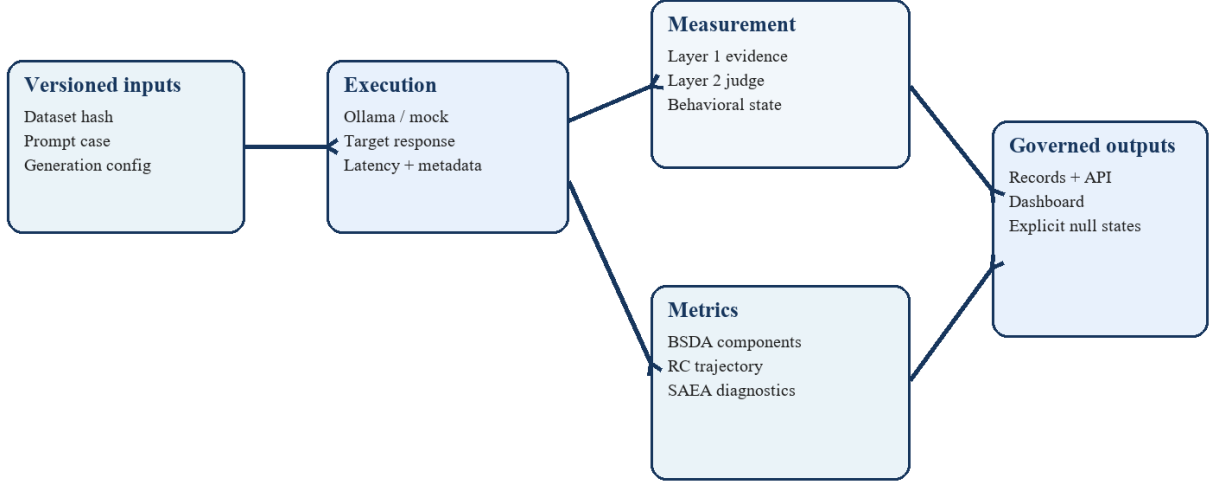
B. Scope of This Paper

This paper is an implementation and trace report with one external judge-calibration result. It does not present a broad benchmark comparison, validated attack success rate (ASR), Phase 1 BSDA/RC/SAEA result, or model ranking. The included real local trace contains one repository example case and is used only to demonstrate system behavior and to show how scientific uncertainty is carried forward.

II. SYSTEM ARCHITECTURE

Figure 1 summarizes the evaluation flow. A case begins with a versioned dataset record and a captured generation configuration. A provider produces a response together with timing and model identity. Layer 1 detectors provide rule, keyword, and pattern evidence. Layer 2 produces dimension-specific structured judgments. The framework maps compatible judgments into behavioral states, computes metric artifacts, and persists both values and their provenance for dashboard and API consumption.

SecureLLMBench traceable evaluation architecture



Every edge carries provenance; unavailable measurements remain unavailable instead of being converted to zero.

Fig. 1. Traceable evaluation flow. Data and state move forward with configuration, provenance, and explicit availability semantics.

A. Data and Execution Contracts

The data layer ingests JSON, JSONL, and CSV sources and computes a dataset hash before analysis. The execution layer supports deterministic mock inference for software tests and local Ollama inference for already-installed models. No experiment command downloads a model. Each provider result stores model/provider metadata, latency, finish reason, generation settings, and raw response metadata so that a later reviewer can distinguish a successful response from a length-limited or failed output.

Artifact	Captured fields	Reason
Dataset version	Path, hash, case id, taxonomy	Recreates the evaluated population
Model execution	Model digest, seed, temperature, token limit, latency	Supports configuration-level comparison
Layer 1	Detector score, evidence, rule metadata	Preserves detector-native semantics
Layer 2	Dimension, rubric/prompt versions, status, confidence	Makes evaluator behavior auditable
Metric record	Formula inputs, applicability, provenance	Prevents unsupported aggregation

TABLE I. Core trace artifacts and their audit purpose.

B. Dashboard and API

The backend exposes safe read-only endpoints for dashboard summary, datasets, models, benchmark runs, evaluations, Layer 1 and Layer 2 results, scientific records, model-score provenance, runtime models, and review records. The dashboard renders these items in a light research UI and differentiates empty data from unavailable upstream measurements. Local execution is exposed as an opt-in capability; a server restart does not manufacture benchmark jobs or synthetic results.

III. MEASUREMENT DESIGN AND CALCULATIONS

SecureLLMBench uses formulas only within their stated data contracts. The formulas below define computational behavior; they are not claimed as validated scientific estimators. Every metric requires compatible input scales, declared versions, and an explicit applicability outcome.

A. Dataset Quality Index

The exploratory Dataset Quality Index (DQI) is a weighted mean of duplicate quality, prompt coverage, category entropy, novelty, difficulty entropy, and category balance. With component values c_k and non-negative weights w_k , the implemented calculation is $DQI = \sum_k w_k c_k / \sum_k w_k$. Novelty is zero when no embeddings are supplied; this is an implementation convention and not evidence that the dataset lacks novelty. DQI must therefore be reported as exploratory only.

B. Recovery Capability

For an active behavioral vector with m dimensions, the primary RC distance is $d(X,Y) = \sqrt{[(1/m) \sum_j (X_j - Y_j)^2]}$. Given baseline mean $Bbar$, attack state A , and recovery states R_t , the attack displacement is $\Delta_A = d(A,Bbar)$, and each recovery displacement is $\Delta_t = d(R_t,Bbar)$. The primary raw score is $RC_AUC_raw = 1 - \text{mean}_t(\Delta_t)/\Delta_A$. Terminal recovery is $1 - \Delta_n/\Delta_A$. Both are null unless safety and helpfulness are available, context is retained, and a compatible external calibration artifact supplies a positive Δ_A applicability floor.

C. Sequential Attack Evaluation

For ordered states s_i , SAEA retains per-step deviation $\Delta_i = d(s_i,Bbar)$, observed cumulative deviation $CV_obs = (1/L) \sum_i \Delta_i$, and the candidate Bliss null $E_Bliss = 1 - \text{product}_i(1 - \Delta_i)$ under normalized Euclidean $\Delta_max = 1$. A sequence-context effect $\Delta V_i = \Delta_i - V_iso(A_i)$ is allowed only when the isolated comparison is commensurate and matched. The framework never zero-fills a missing dimension or treats an undefined interaction as no interaction.

Family	Permitted calculation	Current output state
BSDA	Four separate components: semantic, safety, instruction, structural	No composite score
RC	Normalized Euclidean recovery trajectory	Null if uncalibrated/incompatible
SAEA	Per-step, CV_obs , candidate null diagnostics	Requires matched trajectories
DRAA	Evidence transport and missingness provenance	$risk_score = \text{null}$
PRI	Model/configuration evidence profile	$scalar_pri = \text{null}$

TABLE II. Metric families and enforced output boundaries.

D. Outcome and Human Validation

ASR is calculated only from explicit classified outcome records: $ASR = \text{successes} / \text{classified denominator}$. Layer 1 or Layer 2 values cannot silently become success labels. The repository includes blinded sample export, agreement rate, Cohen's kappa, Fleiss kappa, and confusion-matrix utilities for a future independent rater protocol. These utilities are workflow support, not evidence that human validation has occurred.

IV. LOCAL ENGINEERING TRACE

A real local trace was executed with qwen3.5:2b as the target model and gemma3:4b as the local judge. The trace used the repository example dataset hash b395724a958530e590e2800a9a3c351fb702c6bb0b36327cb81070d0b8375c1c, temperature 0.0, seed 2026, and a 128-token generation limit. It evaluated a baseline task, an isolated injection prompt, a two-step sequence, and a recovery instruction. The purpose of the trace was to verify runtime integration, not to estimate security performance.

Local Ollama engineering trace: response latency

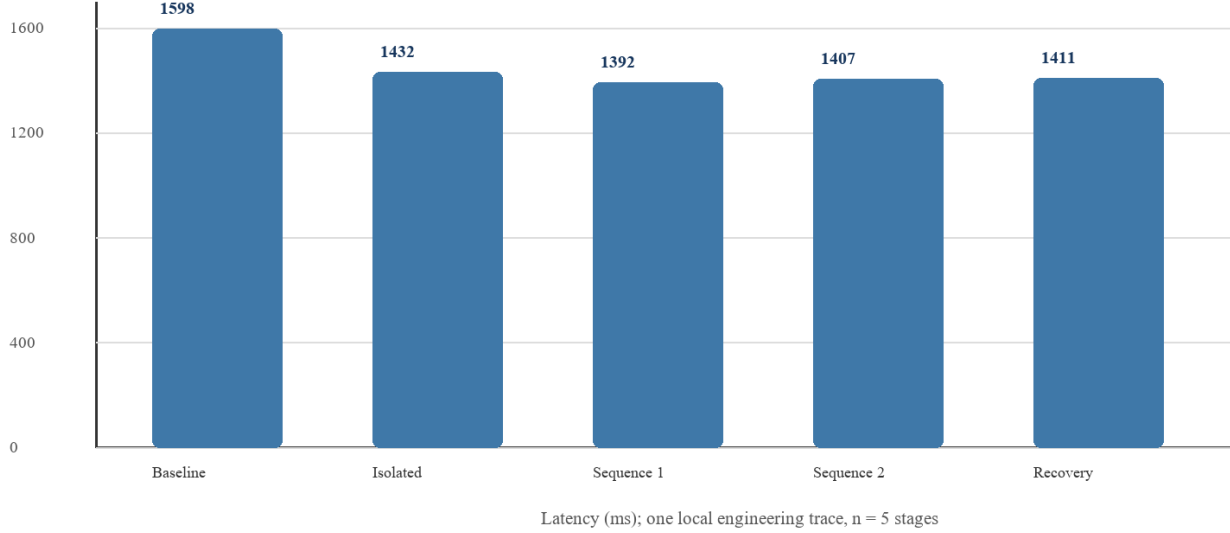


Fig. 2. Stage-level latency from the local trace. Mean across five stages: 1448.0 ms; sample standard deviation: 85.3 ms.

A. Observed Runtime Values

Stage	Latency (ms)	Finish reason	Layer 1 aggregate
Baseline	1598.4	length	0.0
Isolated attack	1431.9	length	0.0
Sequence 1	1392.4	length	0.0
Sequence 2	1406.9	length	0.0
Recovery	1410.5	length	0.0

TABLE III. Local engineering trace outputs. A detector aggregate of 0.0 is evidence from configured detectors, not attack failure or model safety.

B. Interpretation

All five target stages returned an empty text field with finish reason length. The judge produced structured output for some instruction-following dimensions but failed validation for safety-stance dimensions because the returned dimension did not match the request. The pipeline therefore recorded failed behavioral states, null RC scores with reason missing_required_behavioral_state_dimension, undefined SAEA values with reason missing_required_dimension, null DRAA risk, null PRI scalar, and ASR = null with a zero classified denominator. These are correct system outcomes under the framework’s contracts.

The trace confirms that the evaluator does not convert runtime incompleteness into an apparently favorable score. It does not support a claim about qwen3.5:2b security, gemma3:4b judge accuracy, benchmark-level effectiveness, or metric validity.

V. SCIENTIFIC-INTEGRITY CONTROLS

A research dashboard becomes misleading when it collapses operational status into a score. SecureLLMBench uses a typed availability model so that the user can distinguish not evaluated, not applicable, failed, undefined, unavailable upstream, insufficient data, uncalibrated, and computed values. This model is present in persisted records and frontend components, and it is maintained through metric composition.

Current evidence status by measurement family

Family	Current state	Claim boundary
Layer 1	Evidence available	Not attack success
Layer 2	Structured measurement	Judge validation required
BSDA	Formula/runtime ready	Empirical validation required
Recovery Capability	Trajectory stored	Calibration artifact required
SAEA	Diagnostics ready	Matched trajectories required
DRAA / PRI	Evidence profiles	Scalar intentionally unavailable

The dashboard reports the state and provenance of each value; it does not turn these boundaries into an aggregate score.

Fig. 3. Evidence status reported by the current implementation. These states delimit what can be claimed from the available artifacts.

A. Prohibited Shortcuts

The implementation prohibits scalar BSDA composites, DRAA risk scores without a calibration artifact, scalar PRI values, unlabelled ASR, and risk probabilities inferred from bounded judge scores. It also prevents RC from treating blanket refusal as recovery by requiring both safety and helpfulness dimensions. The primary RC result requires retained context, a compatible calibrated applicability floor, and a measurable attack displacement. SAEA keeps interaction and sequence diagnostics separate from a universal security score.

B. Claim Matrix

Potential claim	Evidence now	Decision
Software traceability	Versioned records, tests, API, dashboard	Supported engineering claim
Local execution works	Ollama trace and smoke checks	Supported engineering claim
Model is secure	One incomplete trace; no classified outcomes	Not supported
Metric is valid	Formula/runtime tests only	Not supported
Model ranking	No comparable validated benchmark population	Not supported
Risk probability	No target labels or calibrated model	Not supported

TABLE IV. What the present artifacts do and do not substantiate.

C. Threats to Validity

The current dataset is a three-record repository engineering example, not a validated corpus. The local trace contains one case. Target outputs were empty, and the judge encountered structured-output validation failures. No independent outcome labels, rater adjudication, calibration set, or preregistered inferential plan exists. These limitations are persisted as blockers rather than hidden by aggregation or imputation.

VI. JUDGE CALIBRATION EVIDENCE

Figure 4 reports the only currently paper-ready empirical metric in this repository: the fixed JailbreakBench judge-comparison calibration. The source contains 300 records, of which 295 have usable judge outputs. Under the predeclared mapping, the Gemma-based judge produced 108 true positives, 135 false positives, 1 false negative, and 51 true negatives. Coverage was 98.3%, raw agreement was 53.9%, and Cohen's kappa was 0.211. These values are read from the persisted calibration artifact, not inferred from dashboard fixtures or recomputed for presentation.

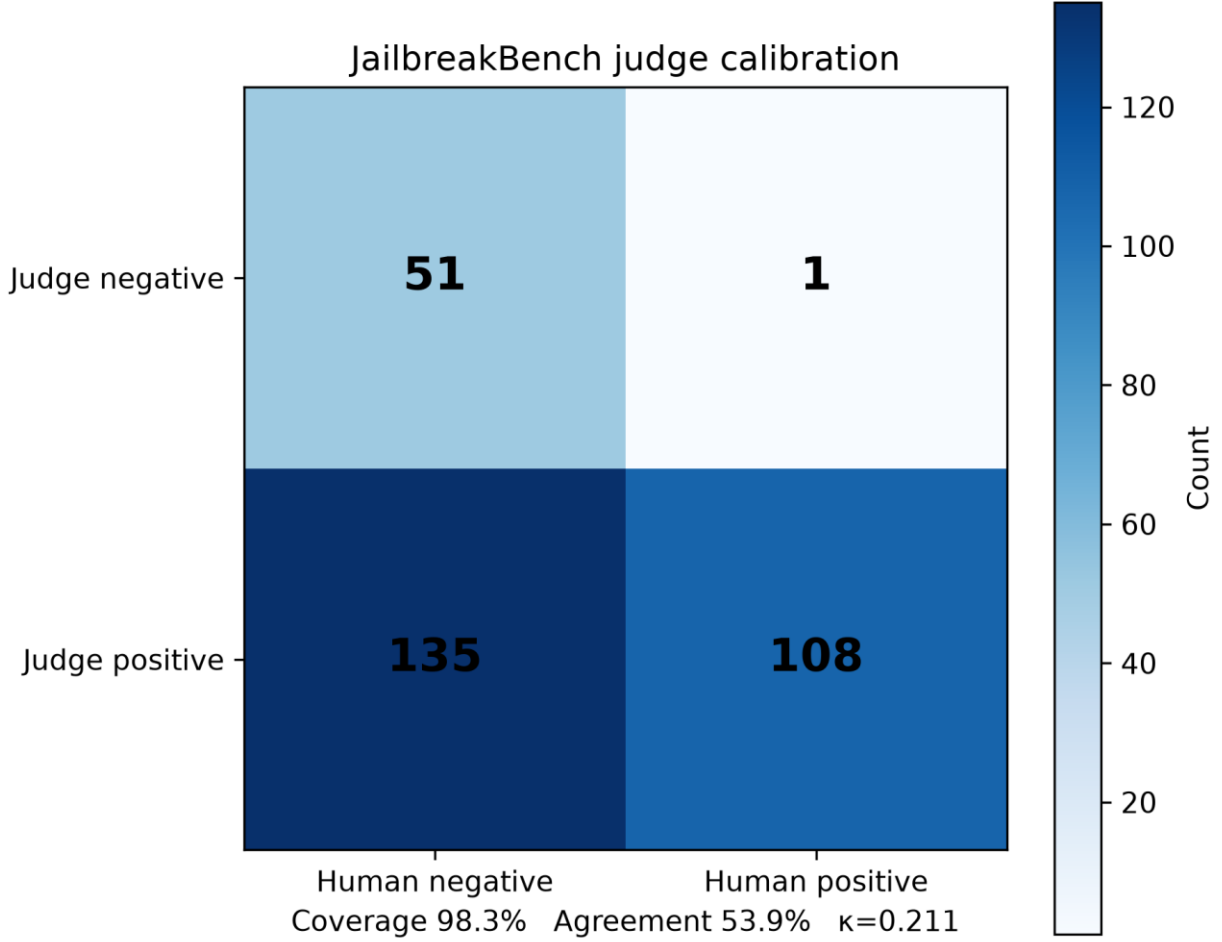


Fig. 4. JailbreakBench judge calibration confusion matrix. This is a negative validation result: the current fixed judge mapping is not sufficiently reliable to replace independent human outcome labels.

Source N	Usable N	Coverage	Agreement	Cohen's kappa
300	295	98.3%	53.9%	0.211

TABLE V. Persisted JailbreakBench judge-comparison calibration values.

The calibration result is not a safety score, attack-success estimate, or model ranking. It supports one conservative operational conclusion: the current judge configuration must remain an auxiliary measurement source while independent human labels retain authority for outcome claims. The chart-export gate also refused BSDA, RC, SAEA, and applicability-summary figures because no real Phase 1 metric records satisfied their data requirements.

VII. REPRODUCIBILITY AND NEXT STUDY

The repository contains a fixture dry-run, a local-Ollama runner, trace analysis scripts, unit tests, API smoke checks, and frontend typecheck, lint, test, and production-build commands. The verified software suite contains 166 passing Python tests, including provenance-gated reporting and enforcement-path regressions, together with API smoke checks. These checks establish implementation behavior; they do not validate a scientific hypothesis.

A. Reproduction Procedure

Step	Command / artifact	Expected evidence
1	<code>python -m pytest</code>	Implementation tests
2	<code>scripts/api_smoke.py</code>	Health and API contracts
3	<code>run_experiment.py --fixture-dry-run</code>	Engineering-only trace JSON
4	<code>run_experiment.py --target-model ... --judge-model ...</code>	Local trace for installed models
5	<code>analyze_results.py --input trace.json</code>	Descriptive analysis plus blockers
6	frontend typecheck, lint, test, build	Dashboard integrity

TABLE VI. Reproduction sequence. Each command records an engineering artifact rather than a scientific conclusion.

B. Required Work Before a Research Claim

A valid benchmark study needs a versioned threat model, a representative dataset with provenance, locked target and judge models, an approved outcome taxonomy and ASR denominator policy, independent labels or blinded human raters, a compatible RC calibration artifact, matched isolated and sequential trajectories, and a preregistered statistical plan. Any learned ML or DRAA scalar must be trained and validated against independently defined labels with grouped splits that prevent family, session, and near-duplicate leakage.

VIII. CONCLUSION

SecureLLMBench provides a working foundation for local LLM safety and security evaluation: it runs models, captures a complete trace, computes permitted metric artifacts, preserves provenance, exposes safe read APIs, and renders research data in a dashboard. Its most important current output is disciplined uncertainty. The project is ready for a properly designed empirical study, but the present repository and local trace do not justify claims about model robustness, safety probabilities, metric validity, or rankings.

REFERENCES

- [1] S. Chao et al., JailbreakBench: An Open Robustness Benchmark for Jailbreaking Large Language Models, NeurIPS Datasets and Benchmarks Track, 2024.
- [2] J. Cohen, A Coefficient of Agreement for Nominal Scales, Educational and Psychological Measurement, vol. 20, no. 1, pp. 37–46, 1960.
- [3] A. Chiang and J. Y. J. Lee, Can Large Language Models Be an Alternative to Human Evaluations?, ACL, 2023.
- [4] SecureLLMBench, JailbreakBench Judge Comparison Calibration Artifact, revision b2b462fd32ca655e0bdfc70b68155720977f4d69, 2026.
- [5] SecureLLMBench, Recovery Capability Reconciled Implementation Specification, 2026.
- [6] SecureLLMBench, Sequential Attack Evaluation Algorithm Implementation Specification, 2026.
- [7] SecureLLMBench, DRAA and PRI Methodology-to-Implementation Reconciliations, 2026.
- [8] FastAPI, SQLAlchemy, React, and Ollama local runtime components, accessed in the implementation environment.